

Shadow: A Cryptographically Verifiable Evidence and Spatial Audit System for AI-Assisted Decisions

From Multi-Agent Answers to Independently Verifiable AI Decision Evidence

Alex Ji

2026-07-21

Contents

Abstract	2
1 Introduction	2
2 Project Evolution	3
3 Problem Statement	3
4 Objectives and Research Questions	4
5 Background and Related Work	4
6 System Requirements	5
7 System Architecture	5
8 Evidence and Trust Model	6
9 Claim–Evidence Graph	7
10 Third-Party and LLM Output Audit	7
11 User Experiences	7
11.1 Browser verifier (host-tested · browser-rendered · browser-recorded)	7
11.2 Unity Shadow Lens (unity-authored · Android-built · device-validation-pending) .	7
11.3 Three.js spatial replay (browser-rendered · research prototype)	8
11.4 Self-contained explainers (host-tested)	8
12 Implementation	8
13 Evaluation	8
13.1 Host tests	8
13.2 Browser acceptance	9
13.3 Android build	9

13.4 Unity tests	9
13.5 Three.js prototype	9
13.6 Tamper case study	9
14 Results	10
15 Limitations	10
16 Future Work	10
17 Conclusion	10
18 Appendices	11
19 Figures	11

Abstract

AI-assisted decision systems increasingly produce conclusions that a person must review and sign off on, yet the answer such a system returns is not the same as verifiable evidence of how it was produced. Ordinary application logs are mutable and platform-specific, and a model’s own explanation is generated prose rather than a checkable record; neither lets an independent reviewer confirm what happened or detect after-the-fact tampering. This capstone presents **Shadow**, an independent evidence and decision-audit layer for AI systems. Shadow represents an AI-assisted workflow as a canonical, deterministic sequence of structured events, links them in a SHA-256 hash chain, and seals the batch with an Ed25519 signature, producing a portable evidence bundle that a third party can verify — including offline in a browser. When a record is altered, the verifier localizes the exact first failed sequence and flags every downstream event the break invalidates. The system carries three domain profiles (banking, data science, coding agent) under one verification grammar, a bilingual (English / Simplified Chinese) verifier, spatial-replay prototypes in Unity and Three.js, and a built Android mock. The implementation is host-tested (**1,824 of 1,827 tests passing**, 3 environment-gated skips) with browser-acceptance validation in Chromium; the spatial surfaces are prototypes and the Android build is not device-validated. One principle governs the design: **cryptographic verification establishes integrity, not correctness** — it cannot show that a cited source was truthful or that a conclusion was right, and the system states this at every surface. The project’s major limitations are that signing is fixture-grade and device validation and a spatial-comprehension user study are pending; no correctness or compliance claim is made. Capstone II will pursue device validation, a controlled user study for the spatial interface, and the production signing and governance path.

1 Introduction

AI-assisted decision-making is moving from novelty to infrastructure. Systems built on large language models now draft credit memos, triage documents, size positions, and recommend actions that a person then signs off on. As these systems take on higher-stakes work, a gap becomes visible: the answer an AI produces is not the same thing as *evidence of how it was produced*, and the two are routinely conflated.

An ordinary application log records that something happened, but it is platform-specific, mutable,

and not designed for an outside party to check. A model’s own explanation of its reasoning is a generated narrative, not a record — it can be fluent and still be unfaithful to what the system actually did. Neither gives a downstream reviewer — an auditor, a compliance officer, a counterparty — a portable artifact they can inspect independently and, crucially, detect tampering in.

This report describes **Shadow**, an independent evidence and decision-audit layer for AI systems. Shadow’s research and engineering objective is narrow and deliberate: produce a **portable evidence record** for an AI-assisted workflow that an independent party can verify — the sequence of events, the sources referenced, the tool and model actions, the hash-chain continuity, the digital signature, and, if the record was altered, the *exact* point of failure and the downstream events it invalidates.

A principle runs through the entire system and through this report: **integrity is not correctness**. Cryptographic verification can establish that a supplied evidence record matches what was sealed. It cannot establish that the underlying source was truthful or that the analytical conclusion was right. Shadow keeps these separate on purpose, and this report is careful never to let one stand in for the other.

2 Project Evolution

This capstone’s current subject, Shadow, grew out of an earlier project, **Orallexa** — a multi-agent AI decision-support system in which several LLM “voices” (a bull, a bear, a judge, and a critic) debated a question and produced a recommendation. Orallexa worked, and building it surfaced the more important research problem.

The lesson from the multi-agent work was that producing *more opinions* is not the same as producing *trust*. A panel of AI voices can be persuasive and still leave a reviewer with no way to check what sources or actions actually underpinned the conclusion. The debate is an input to a decision; it is not evidence of one. Adding a fifth voice made the answer richer, not more verifiable.

Shadow is the refinement of that problem. It shifts the emphasis from generating answers to **recording verifiable evidence** of an AI-assisted workflow — events, sources, tool actions — in a form that is cryptographically sealed and independently checkable. The multi-agent analysis is retained as a *domain capability* (Shadow ships a deterministic five-voice loan council), but it is deliberately **not** the foundation of trust. Trust rests on the evidence layer beneath it, not on the persuasiveness of the voices above it.

This is a sharpening of the original research question, not an abandonment of it. Orallexa asked “how do we get a better AI answer?” Shadow asks “how do we let someone verify what the AI actually did?” — which is the harder and more durable problem.

3 Problem Statement

The central problem this project addresses is:

How can an AI-assisted workflow produce a **portable evidence record** that allows an independent party to verify the event sequence, the source references, the tool and model actions, the hash-chain continuity, and the digital signature — and, when the record has been altered, to identify the exact first failed sequence and the downstream events

it affects — **without claiming that cryptographic verification proves analytical correctness?**

The final clause is not a caveat bolted on at the end. It is part of the problem. A system that lets integrity masquerade as correctness would be worse than no system, because it would manufacture confidence it has not earned. The design goal is verifiable integrity *with an explicit, defended boundary* around what that integrity does and does not prove.

4 Objectives and Research Questions

The project’s objectives are:

1. Represent an AI-assisted workflow as a canonical, deterministic sequence of structured events.
2. Link those events cryptographically (hash chain) and seal them with a digital signature.
3. Provide an independent verifier that can validate the record — including offline — and localize any tamper.
4. Support multiple domain profiles under one verification grammar.
5. Provide both a precise 2D audit surface and spatial (3D/XR) replay prototypes, without letting the spatial layer become required for verification.

These objectives are examined through four research questions.

- **RQ1 — Tamper localization.** Can Shadow detect the modification of an earlier AI-workflow event and identify the first failed sequence number?
- **RQ2 — Cross-surface verification.** Can the same evidence semantics be verified across command-line, browser, and spatial (Unity / Three.js) surfaces?
- **RQ3 — Separation of concerns.** Can source, claim, and workflow provenance be represented *without collapsing integrity, stance strength, and analytical correctness into a single status*?
- **RQ4 — Spatial comprehension.** Can a spatial replay interface improve a reviewer’s understanding of provenance and tamper propagation while retaining a precise 2D fallback? **RQ4 is not yet user-studied**; the spatial surfaces exist as prototypes, and the comprehension claim is a Capstone II research question, not a present result.

5 Background and Related Work

Shadow draws on several established areas.

Cryptographic hash chains. Linking records so that each entry commits to the previous one — so that altering an earlier entry invalidates everything after it — is the mechanism behind tamper-evident logs and Merkle-linked structures. Shadow uses a per-event hash chain sealed by a batch root, following the general construction described in cryptographic literature and standardized primitives (NIST FIPS 180-4 for SHA-256).

Digital signatures. Shadow signs the sealed record with Ed25519 (RFC 8032). A signature establishes that the holder of a private key attests to the sealed bytes; it does not establish anything about the truth of the content those bytes describe.

Provenance and data lineage. The idea that a result should carry a checkable record of the sources and transformations that produced it is long-standing in data-management and scientific-

workflow research (e.g., the W3C PROV data model). Shadow applies this to AI workflows specifically, where the “transformations” include model and tool actions.

AI observability and model governance. Recent industry tooling records agent traces and telemetry (e.g., OpenTelemetry-style spans). Shadow is complementary rather than competing: it can ingest a third-party agent’s trace structurally and *seal* it, but the semantic audit of ingested content is a separate, still-developing capability (see §10). Governance frameworks — U.S. supervisory guidance and EU regulation — are discussed in §8 with an explicit note on scope.

Claim–evidence structures. Representing an assertion together with its supporting and contradicting evidence and its sources — rather than as free prose — is more inspectable than a narrative answer. Shadow implements a claim–evidence graph as a shared fact source.

Spatial visualization and audit replay. Immersive and spatial presentation of abstract structures is an active visualization research area; the open question, which this project treats honestly, is *whether* spatial presentation measurably helps for this task — a question RQ4 defers to a user study.

Web sources cited in this section were accessed 2026-07-21. Standards (FIPS 180-4, RFC 8032, W3C PROV, Regulation (EU) 2024/1689) are cited to their primary publications; see References.

6 System Requirements

Functional requirements. Shadow must: capture structured events; maintain a deterministic sequence; link events cryptographically; sign and seal the evidence; preserve source maps; verify evidence independently; show the exact tamper failure point; support multiple domain profiles; export a portable evidence bundle; and provide both 2D and spatial replay.

Non-functional requirements. Verification must be deterministic and offline-capable. Untrusted evidence must be handled safely (bounded parsing, no code execution, escaped rendering). The verifier must be bilingual (English and Simplified Chinese) with locale parity that never alters a hash, quote, signature, or verdict. Status must be communicated accessibly. The verifier and the self-contained explainers must have **no external runtime dependency**. Trust boundaries must be explicit.

7 System Architecture

Shadow is organized into a core evidence layer, a set of domain profiles, and several interface surfaces of deliberately *different maturity*.

SHADOW CORE

```
canonical evidence representation (spec/evidence-bundle.  
schema.json, v1: bundle_version, spec_version, header,  
events, batch_root, signatures)  
signed hash-chain record → Ed25519 (spec/attestation.  
schema.json) · source maps · verifier  
profile validation · claim-evidence graph  
ingested-output audit (structural; semantic = pending)
```

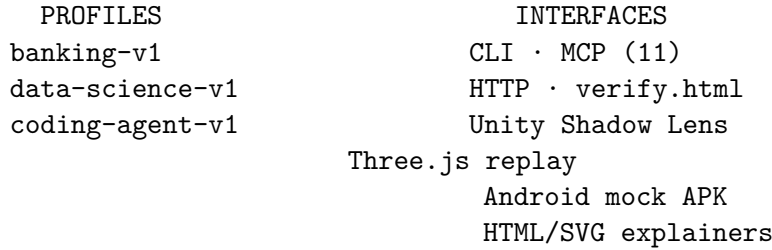


Figure 1. Shadow layered architecture. Mermaid source for regeneration is in Appendix E.

The interfaces are **not** of identical maturity, and the report does not present them as such. The CLI, MCP server (11 tools), HTTP endpoints, and the browser verifier (`verify.html`) are **host-tested** and, for the verifier, **browser-rendered** and **browser-recorded**. The Unity Shadow Lens is **unity-authored** with an **Android-built** mock APK; its on-device behavior is **device-validation-pending** (Beam Pro). The Three.js spatial replay is a **browser-rendered** research prototype. The self-contained explainers are **host-tested** deterministic HTML/SVG.

8 Evidence and Trust Model

Shadow separates concerns that are frequently — and dangerously — merged:

- **Record integrity** — the bytes verify against the sealed record.
- **Hash-chain continuity** — each event commits to the previous one; the chain is unbroken.
- **Digital signature** — the sealed batch root is signed with a known key.
- **Profile validation** — the evidence conforms to a domain profile’s schema.
- **Source resolution** — cited sources resolve to referenced material.
- **External anchor** — optional third-party timestamp/inclusion (not required for the base guarantee).
- **Analytical correctness** — whether the conclusion is *right*. **Shadow does not establish this.**

Stated prominently, because it is the load-bearing claim of the whole project:

Cryptographic verification can establish that the supplied evidence matches the sealed record. It does not establish that the underlying source was truthful or that the analytical conclusion was correct.

Signing status is also kept honest. The current release uses **fixture signing**, not production signing. The verifier’s “Verify the Verifier” view reports either `ASSETS MATCH SIGNED MANIFEST` or `INDEPENDENT COMPARISON NOT PERFORMED` — a page hashing *itself* is explicitly **not** presented as trust. Independent trust requires comparing the page’s asset hashes against a signed release manifest obtained through a separate channel and comparing the release-key fingerprint out of band. **No production signing key exists in this project**, and none was generated for it.

9 Claim–Evidence Graph

Rather than emit a prose answer, Shadow can represent a conclusion as a structured graph (`lib/claim-evidence-graph.mjs`, fixture version `shadow-claim-evidence-graph/1.0`): each **claim** carries its **supporting evidence**, any **contradictory evidence**, its **source references**, and a **graph hash**; the structure also surfaces **unsupported claims** and **unresolved sources**.

This is more inspectable than prose because the gaps are explicit — a reviewer can see *which* claims lack support and *which* sources did not resolve, rather than having to trust a fluent paragraph. It does **not** prove truth. A well-formed graph with resolvable sources can still rest on a false source; the graph makes that checkable, not impossible. This capability is **source-authored** this slice (the shared fact source for Shadow, Unity, and the verifier) and is exercised by `test/shadow-claim-evidence-graph.test.js`.

10 Third-Party and LLM Output Audit

Shadow defines an ingestion boundary for output produced by a third-party agent or LLM, with a status model: `ACCEPTED`, `ACCEPTED_WITH_WARNINGS`, `QUARANTINED`, `REJECTED`, `NOT_AUDITED`. The boundary applies schema validation, attempts source/citation resolution, flags unsupported claims, and looks for prompt-injection indicators before an ingest decision.

The honest status: the **structural** side — sealing a third party’s trace so its integrity is checkable — is **host-tested**. The **semantic** audit of ingested content (`auditIngestedOutput`), which is the genuine third-party differentiator, is **source-authored** and **device-pending** this slice. This is **not** a complete AI-safety solution and is not presented as one; it is an ingestion-boundary design with a partial, tested structural implementation and a defined semantic layer still under development.

11 User Experiences

11.1 Browser verifier (host-tested · browser-rendered · browser-recorded)

The browser verifier (`verify.html`) offers two modes — **Verify Evidence** and **Verify the Verifier** — in **English and Simplified Chinese**. On a valid bundle it reports the independent statuses; on a tampered bundle it localizes the failure. It carries a signed fixture manifest, enforces a Content-Security-Policy with **zero external runtime requests**, and verifies evidence offline. Its current limitation is stated plainly: full-page cold loading requires initial access to local assets (offline-after-initial-asset-load), and the signing is **fixture**, not production.

Real rendered validation (Playwright, Chromium 149.0.7827.55, viewports 1440×900 / 1280×720 / 390×844) passed the eight interactive flows in both locales, with locale parity confirmed (hashes, IDs, quotes, and sequence numbers unchanged by language) and CSP/network checks passing with no external requests. Screenshots are in `verify-acceptance/screenshots/` (Figures 5–8).

11.2 Unity Shadow Lens (unity-authored · Android-built · device-validation-pending)

The Unity Shadow Lens (Unity **6000.0.23f1**) presents three domain workspaces — Banking, Data Science, and Coding Agent — with a provenance audit arc, source highlighting, and explicit **Verify / Tamper / Reset** actions. Interaction uses **head-directed focus** via an XR gaze-interactor

abstraction — hover and highlight only. This is **not eye tracking**, there is no RGB capture, and there is no dwell-based approval; selection and approval are kept explicitly separate. An Android **mock** APK is built (§13.3).

Honest status: the Shadow Lens is **unity-authored**; individual Unity tests are reported only where actual test-run evidence exists (§13.4). **Beam Pro device validation is pending**. No XREAL native integration or 6DoF is claimed on the mock build.

11.3 Three.js spatial replay (browser-rendered · research prototype)

A browser-accessible Three.js prototype explores four layouts for spatial replay — an audit **arc**, a layered **DAG**, a **timeline**, and a **hybrid 2D/3D** view — over a shared 3D scene contract, with focus-plus-context navigation and a **2D fallback**. It is **browser-rendered** and **browser-recorded**; there is **no user study yet**, so no comprehension benefit is claimed (RQ4).

11.4 Self-contained explainers (host-tested)

Deterministic HTML/SVG audit-chain animations, bilingual, honoring reduced-motion preferences, with **no third-party runtime dependency**. Their didactic message is the project’s thesis in miniature: **integrity does not equal correctness**.

12 Implementation

Shadow’s core is implemented in **Node.js / JavaScript** with **JSON Schema** validation; the cryptography uses **Ed25519** (RFC 8032) for signatures and **SHA-256** (FIPS 180-4) for the hash chain, over a **canonical serialization** so that the same logical record always hashes identically. The browser verifier uses **WebCrypto** and runs offline under a strict CSP. The spatial surfaces use **Unity 6 / C#** (Shadow Lens) and **Three.js** (web replay). Browser acceptance is driven by **Playwright**; the Android mock is built through the Unity **Android / Gradle / IL2CPP** pipeline. Integration surfaces include an **MCP** server exposing 11 tools. The explainers are plain **HTML / SVG / CSS**.

Code snippets are used sparingly in this report; data-flow and architecture diagrams (Figures 1–2) carry the design, and the repository is the implementation source of truth.

13 Evaluation

13.1 Host tests

Running the host test suite at the audited commit (5106799) on 2026-07-21 gives **1,824 of 1,827 tests passing, 0 failing, 3 skipped**. The three skips are environment-gated (a Mistral OCR key and other live-network/OpenSSL gates), not disabled coverage. (The repository’s pinned reconciliation file **release-state.json**, last generated at commit 10909ed, records 1,595/1,598; it trails the current branch, and the figure reported here is the live re-run.) A drift guard (**test/product-facts-drift.test.js**) enforces that **README**, **llms.txt**, **index.html**, and presentation copy agree with **product-facts.json**.

13.2 Browser acceptance

Validated in real Chromium 149.0.7827.55 (Playwright 1.61.1), English and Simplified Chinese, across the eight interactive flows: valid evidence, tampered evidence, Verify-the-Verifier valid, Verify-the-Verifier mismatch, and their locale pairs. Signed assets carried stable content hashes (`verify.html`, `verify/locales/en.json`, `verify/locales/zh-CN.json`). CSP and network checks passed with **0 external requests and 0 violations**; no console errors; responsive at three viewports; evidence verification worked offline. Screenshots: `verify-acceptance/screenshots/`.

13.3 Android build

An Android mock APK is **built** (not device-validated): `mock-stable-5168b07.apk`, **24,442,084 bytes**, SHA-256 `93f2a81aa5f965aec540526abe621b152c7507c03c0fea51d381094bd548d0b8`, produced by Unity 6000.0.23f1. This is a **built** artifact used as the demo baseline; it is **not** called device-validated, and Beam Pro / XREAL on-device behavior remains pending.

13.4 Unity tests

Unity test status is reported only from actual test-run evidence. Authored Unity test scripts exist under the Shadow Lens project; where a test was *executed*, it is labeled unity-tested, and where it is authored but not yet executed on-device it remains unity-authored / device-pending. This report does **not** aggregate authored tests into a passing count.

13.5 Three.js prototype

Four layouts render with deterministic fixture semantics; screenshots and recordings were generated (browser-rendered / browser-recorded). **No user study** has been conducted.

13.6 Tamper case study

A step-by-step pristine-versus-tampered example (deterministic fixture):

1. **Pristine.** A sealed bundle of N events (`bundle_version 1`), each committing to the previous via the hash chain, with a signed `batch_root`. All independent statuses report clean.
2. **Modify.** One earlier event's payload is altered (for example, a source reference on a mid-sequence event).
3. **First failure.** Verification walks the chain and reports the **first failed sequence** — the earliest event whose recomputed hash no longer matches what the next event committed to.
4. **Downstream impact.** Every event after the first failure is flagged as **affected**, because the signature covered a chain that no longer exists — this is stronger than “one bad event.”
5. **Independent statuses hold.** Even in failure, integrity, signature validity, and *analytical correctness* remain **separate** statuses — the tool reports a broken chain without asserting anything about whether the original conclusion was right.

This case study is the strongest demonstration of RQ1 and RQ3 together, and it is the centerpiece of the practice presentation.

14 Results

The project has demonstrated: deterministic evidence generation from a canonical schema; independent verification (CLI, MCP, HTTP, and offline browser); exact tamper localization to the first failed sequence with downstream-impact propagation; multi-profile evidence semantics under one verification grammar (banking-v1, data-science-v1, coding-agent-v1); a bilingual verifier with locale parity; spatial-replay prototypes (Unity Shadow Lens + Three.js); and a working Android mock build pipeline.

The project does **not** claim: production deployment; legal or regulatory compliance; analytical correctness; Beam Pro validation; full XREAL native integration; or real banking-approval automation.

15 Limitations

Stated directly:

- Evidence is **fixture data**; live-provider validation is limited.
- **Production signing is not implemented**; the current manifest is fixture-signed. No production signing key exists.
- **Device validation is pending** (Beam Pro); **XREAL SDK / native APIs** are not integrated.
- **No completed user study** — the spatial-comprehension benefit (RQ4) is unproven.
- **No proof of source truth** and **no proof of analytical correctness** — by design, integrity correctness.
- **No production KMS/HSM**, no key rotation, no durable production storage.
- **No full PII-retention framework** and **no production incident-response process**.
- The **3D usability benefit** still requires evaluation.

16 Future Work

Capstone II. Beam Pro / XREAL device validation; integration of the shared Unity / Three.js scene contract; Audit Arc V2; Eye RGB capture *only if* supported and available on the target path; an OCR-to-source-map pipeline; real device-performance measurement; a user study for RQ4; and evaluation on real sanitized datasets.

Production path. Production release signing; KMS/HSM; key rotation; durable storage; PII governance; a software bill of materials (SBOM); reproducible builds; and an incident-response process.

Research path. Spatial-versus-flat audit comprehension (the RQ4 study); trust calibration; provenance replay; uncertainty and abstention UX; and multilingual verification semantics.

17 Conclusion

Shadow’s contribution is a shift in what an AI system asks of the person on the other side of a decision. Instead of asking an auditor to *trust the AI’s answer*, Shadow gives them a **portable evidence record they can independently inspect and verify** — its sequence, its sources, its signature, and, if it was tampered with, the exact point where the record breaks and everything the

break invalidates. The system draws a firm line around what that verification proves: it establishes integrity, not correctness, and it says so at every surface. That discipline — verifiable evidence with an honest boundary — is the capstone’s core result, and it is the foundation the Capstone II work builds on.

18 Appendices

Appendix A — Implementation status matrix. See `CLAIM_SOURCE_MATRIX.md` (every report claim → repository path, commit, evidence, honest status).

Appendix B — Test inventory. 1,824/1,827 host tests passing (3 skipped, 0 failed), live re-run at commit 5106799 on 2026-07-21; browser acceptance in `verify-acceptance/BROWSER_ACCEPTANCE_REPORT.md`; Unity tests reported per §13.4.

Appendix C — Key schemas. `spec/evidence-bundle.schema.json` (v1: `bundle_version`, `spec_version`, `header`, `events`, `batch_root`, `signatures`); `spec/attestation.schema.json`; `apps/shadow-lens/contracts/shadow-lens-session.schema.json`.

Appendix D — Build information. Frozen mock APK `mock-stable-5168b07.apk`, 24,442,084 bytes, SHA-256 `93f2a81aa5f965aec540526abe621b152c7507c03c0fea51d381094bd548d0b8`, Unity 6000.0.23f1.

Appendix E — Regenerable diagram sources. Mermaid sources for Figures 1–2 and the status ladder.

Appendix F — Terminology. See the terminology table in `SUBMISSION_README.md`. Note: the deterministic council’s per-voice numbers are **persona priors** — “**STANCE STRENGTH**” — not model confidence or probability of correctness.

Appendix G — Branch / commit inventory. Worktree branch `chore/capstone-final-report-and-practice` based on commit 5106799; audit performed 2026-07-21.

19 Figures

The following are **real rendered screenshots** from the repository’s browser-acceptance run (Playwright, Chromium 149.0.7827.55), not mock-ups. The full catalog, including the architecture and evidence-lifecycle diagrams and their regenerable Mermaid sources, is in `FIGURE_INVENTORY.md`.

This is a Capstone I draft. Figures, the claim–source matrix, and references are provided as companion files in the same submission folder. See `SUBMISSION_README.md` for the file map and the one-command regeneration paths.

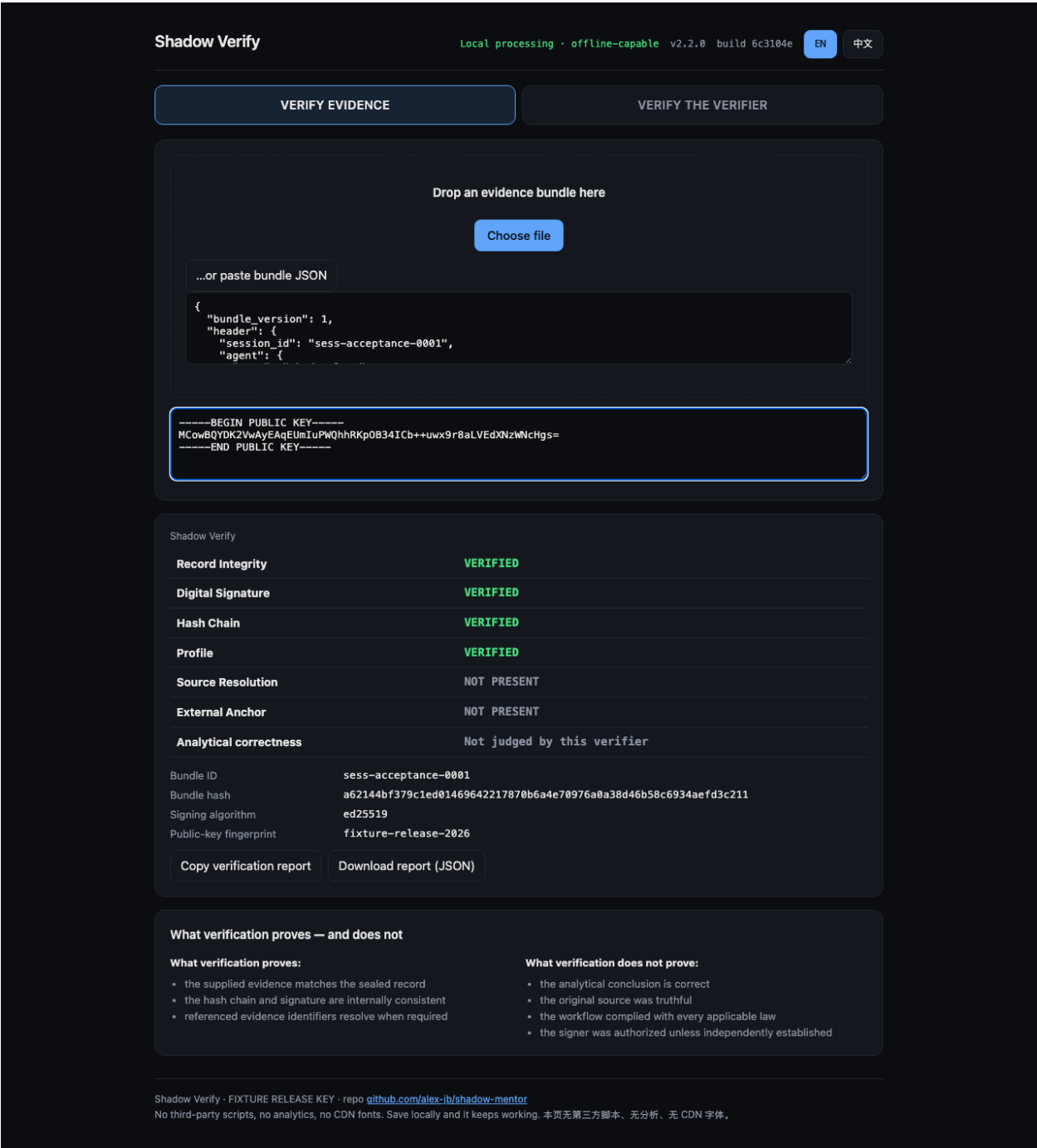


Figure 1: **Figure 5.** Browser verifier — valid evidence (English). Independent statuses reported clean. Status: browser-rendered / browser-recorded. Source: verify-acceptance/screenshots/en-valid-evidence.png.

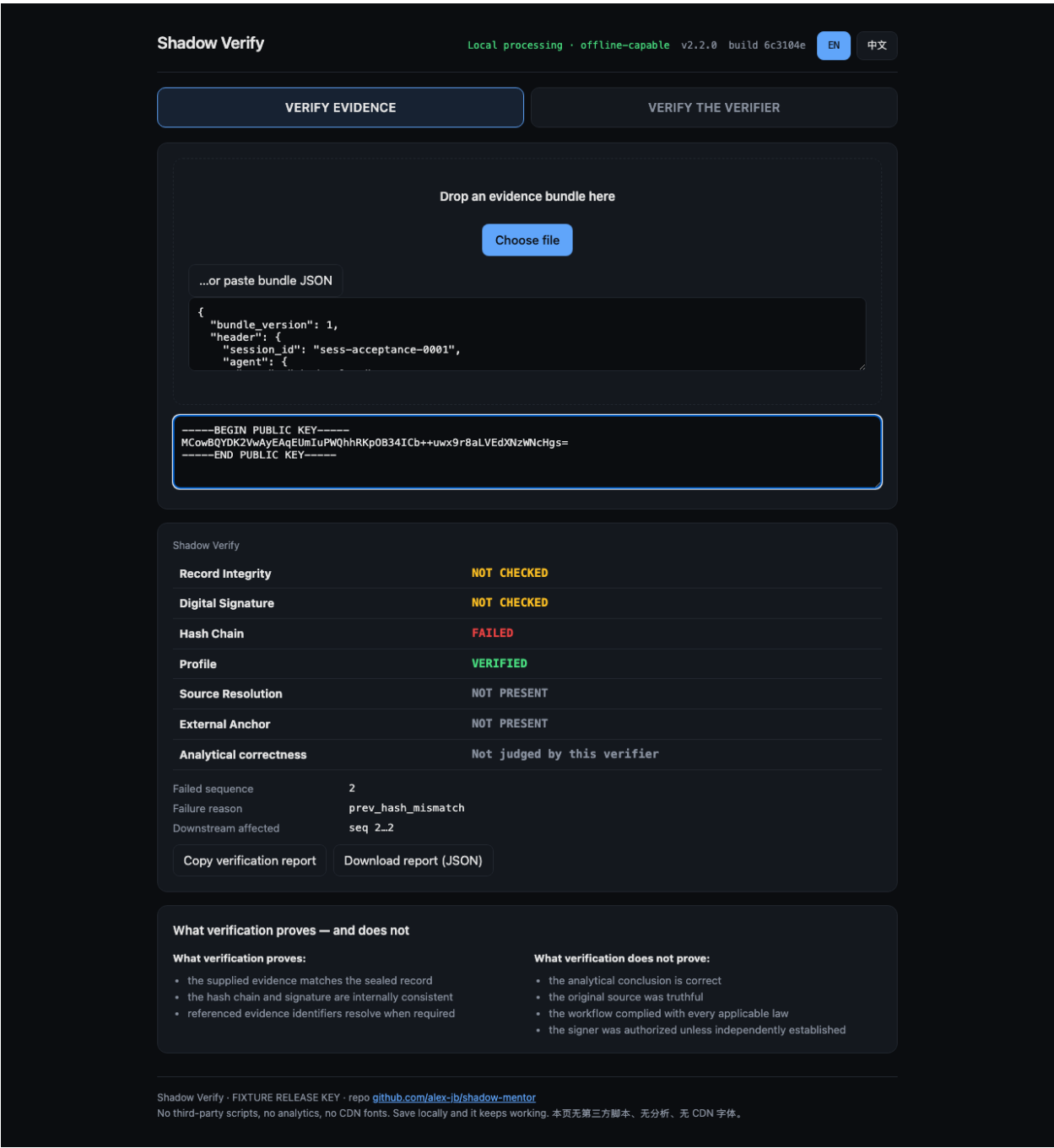


Figure 2: **Figure 6.** Browser verifier — tampered evidence (English). The verifier localizes the first failed sequence and flags downstream-affected events. Status: browser-rendered. Source: `verify-acceptance/screenshots/en-tampered-evidence.png`.

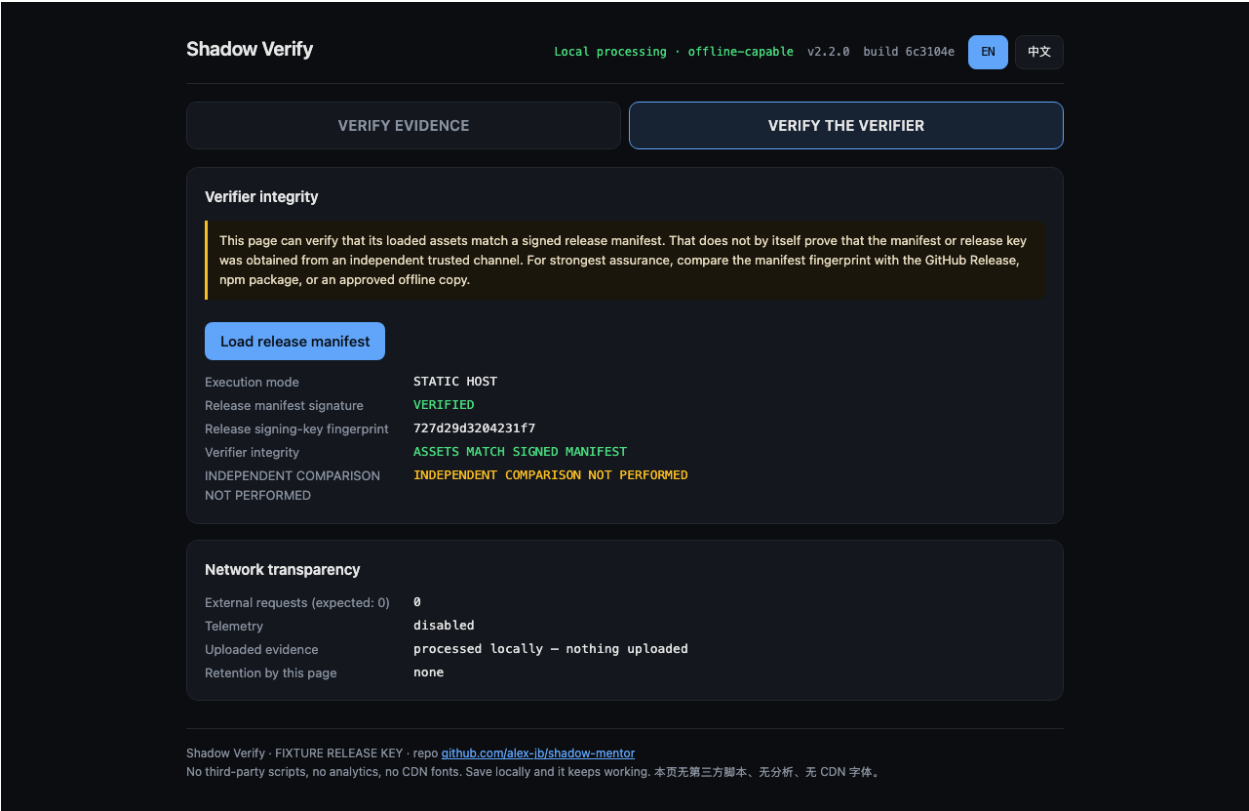


Figure 3: **Figure 7.** Verify-the-Verifier — assets-match-signed-manifest state (English, fixture-signed). A page hashing itself is not presented as trust. Status: browser-rendered. Source: `verify-acceptance/screenshots/en-verifier-valid.png`.

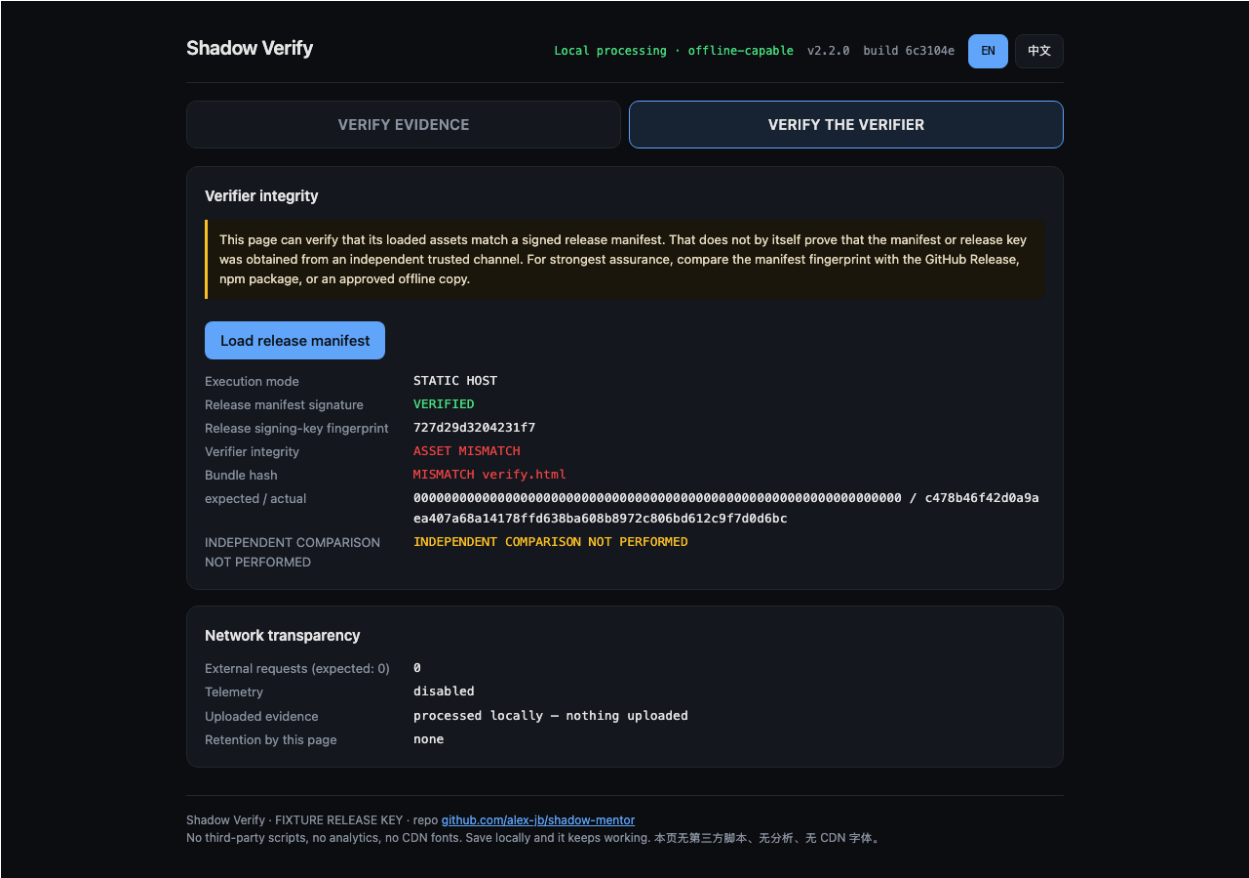


Figure 4: **Figure 7b.** Verify-the-Verifier — asset mismatch state. Status: browser-rendered. Source: `verify-acceptance/screenshots/en-verifier-mismatch.png`.



Figure 5: **Figure 8.** Bilingual parity — valid evidence rendered in Simplified Chinese; hashes, IDs, quotes, and sequence numbers are unchanged by locale. Status: browser-rendered. Source: `verify-acceptance/screenshots/zh-CN-valid-evidence.png`.